Context-First Kanban

Diagram pack for user flow, unlock model, task lifecycle, data flow, architecture, and POC slices

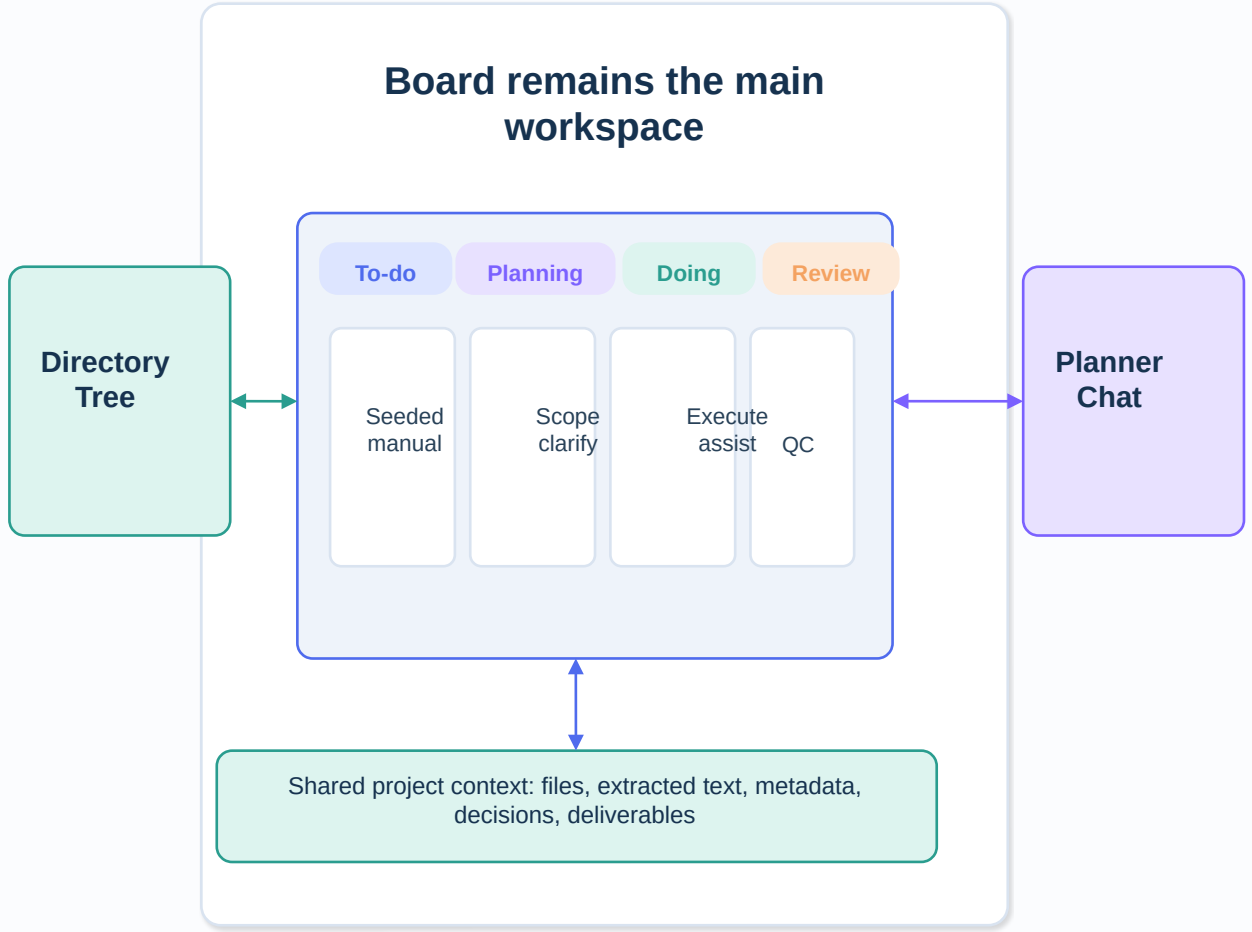
Board-anchored UX | Shared context | AI + human copilot

What this pack is for

These diagrams translate your concept into a dev-facing visual model: how the product unlocks, where AI sits, how tasks move, what data needs to exist, and how a POC can be sliced without losing the core idea.

North-star rule

The board is the main anchoring UX. Directory tree and planner chat are supporting systems around it.



1

End-to-end user flow

The product unlocks in sequence, then settles into board-first execution.

1 **Create project**

Project shell exists but the board is still locked. Initial metadata can be set here.

2 **Context intake**

Only the directory tree is open. User uploads files and adds connectors that explain the project.

3 **Planner conversation**

User explains vision and goals. Agent asks follow-up questions and requests missing documents.

4 **Approve & unlock board**

Planner summarizes the situation and asks for permission to continue. Approved = board unlocks.

5 **Execute through board**

Planner seeds tasks in To-do. User edits tasks. Cards move Planning → Doing → Review → Done/Cancelled.

6 **Close project**

When every task is terminal (Done or Cancelled), the project can be marked complete and closed.

Typical context assets

Weather report, concert news article, supplier quotes, temp worker contacts, lemonade recipes, or any other files/connectors that explain the job.

Key gate

The board remains unavailable until the planner agent and user agree that there is enough context.

Completion rule

Cancelled is a terminal state and counts the same as Done for project-close readiness.

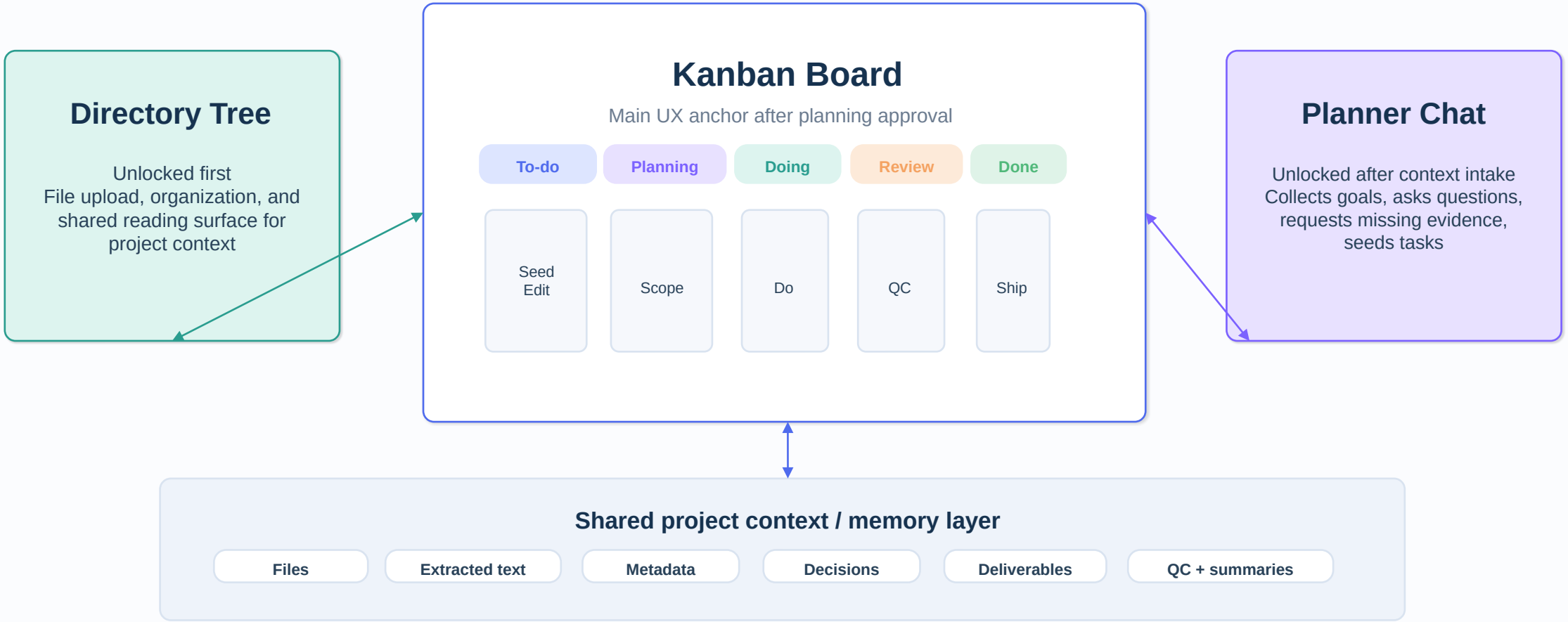
2

Board-anchored workspace model

The board is the day-to-day surface; directory and planner remain attached as context systems.

Operating principle

Humans and AI collaborators read from the same project context and write back into the same project memory. No shadow context.



3

Unlock and permission model

A simple phase gate makes the UX predictable and the implementation easier to reason about.

Project phase	Directory tree	Planner chat	Board	Seed tasks	Close project
Created	Open	Locked	Hidden	Locked	Locked
Context intake	Open	Locked	Hidden	Locked	Locked
Planning	Open	Open	Locked	Locked	Locked
Active board	Open	Open	Open	Action	Locked
Ready to close	Open	Open	Open	Action	Gate
Closed	Open	Open	Open	Final	Final

Design intent

Users are not dropped into an empty board first. They build context first, then move into action once the planner can summarize the job coherently.

Implementation hint

The unlock state can live on the project record as a simple enum plus capability flags. Frontend visibility should derive from that state, not from ad hoc checks.

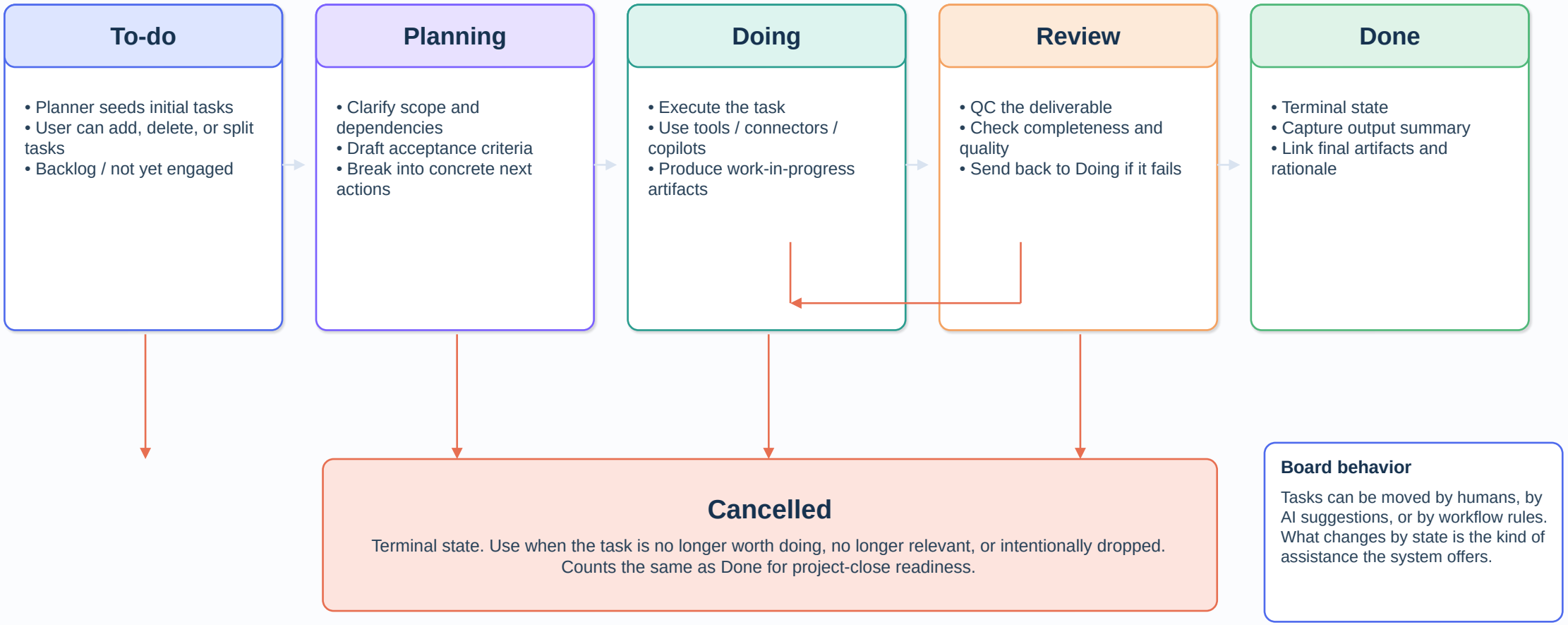
Close rule

Project close is gated by a computed condition: every task must be in a terminal state. Terminal = Done or Cancelled.

4

Task lifecycle and automation points

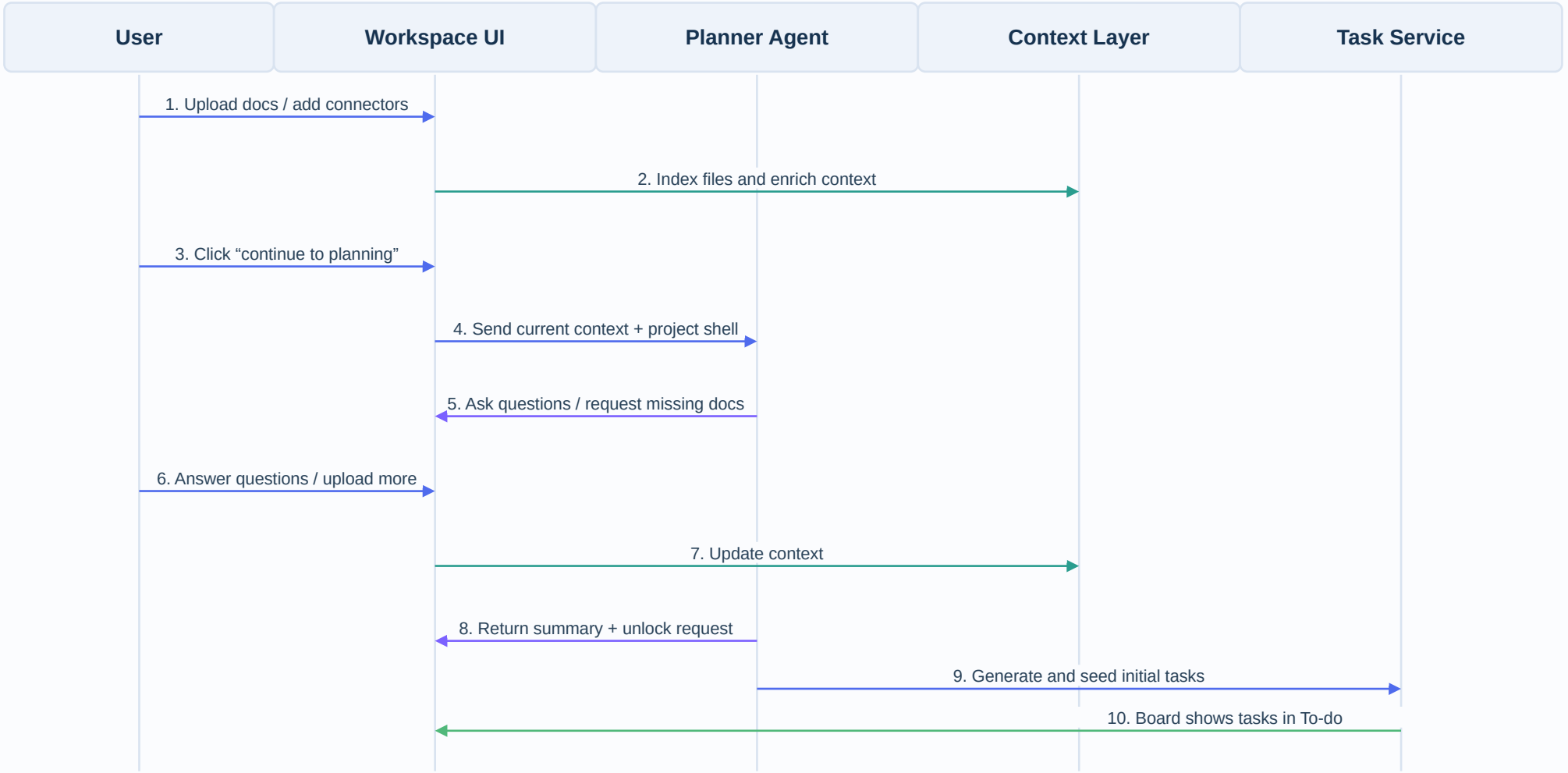
The same card changes meaning by state; different AI routines attach at different stages.



5

Planner-agent interaction sequence

The planner is a gated intake conversation that turns raw context into an executable starting board.



Why the gate matters

The planner does not just answer questions; it decides whether enough context exists to responsibly unlock execution.

Suggested outputs

Planning summary, assumptions, risks, missing info, and a first-cut task list with clear names.

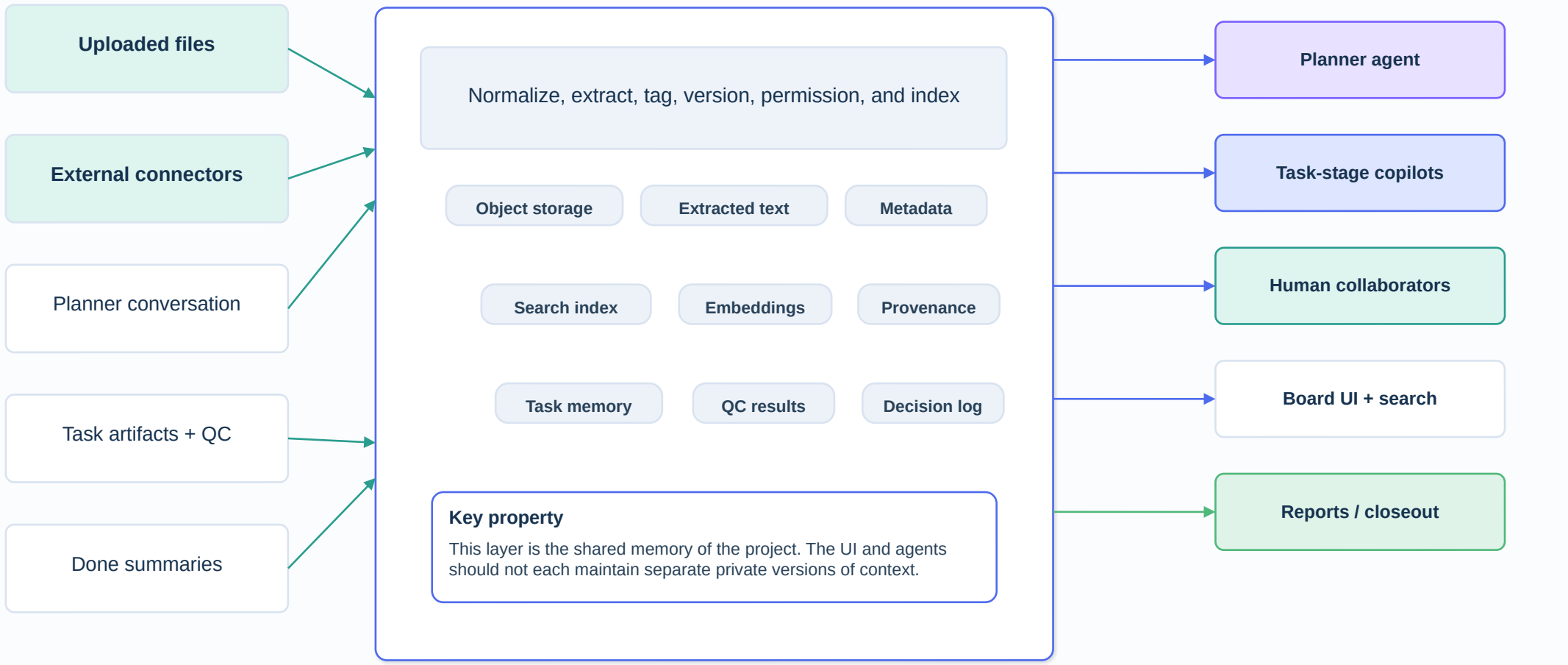
POC simplification

In a POC, planner and task generation can live behind one service boundary even if they later split.

6

Shared context and data flow

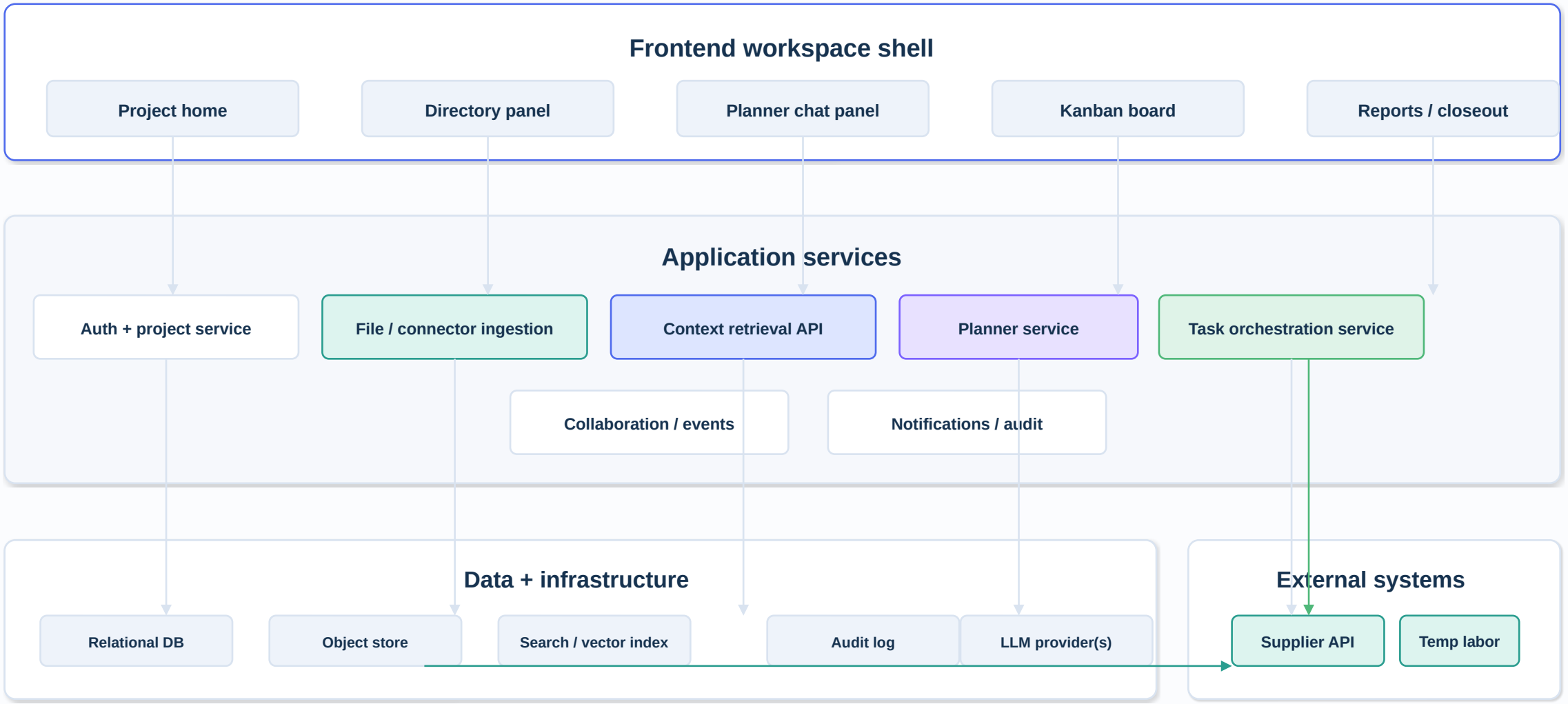
Every project artifact becomes usable context for both people and AI services.



7

Reference architecture for MVP / POC

Modular service boundaries let the product start simple without boxing future evolution in.



Pragmatic POC note

These can start as modules inside one backend as long as the interfaces are clean.

Key non-negotiable

The shared context layer and the board-first interaction shell matter more than microservice purity.

8

Recommended POC build slices

A thin vertical slice sequence keeps the concept testable without building every AI behavior at once.

1. Project shell

- Create project
- Set project status
- Persist unlock state

Foundation

Demo outcome

Project exists.

2. Context intake

- Directory tree only
- Upload files / connectors
- Human-readable shared context

Enable next gate

Demo outcome

Context is visible.

3. Planner gate

- Planner chat
- Ask follow-ups
- Summary + board unlock decision

Enable next gate

Demo outcome

Board unlocks responsibly.

4. Board core

- Seed tasks in To-do
- Move cards across states
- Project close rule

Usable MVP

Demo outcome

Team can execute.

5. Stage copilots

- Planning assist
- Doing assist
- Review QC + done summary

Delight layer

Demo outcome

AI support deepens by stage.

Suggested demo path

Use your concert / lemonade stand scenario with known documents first. It makes the unlock model and shared-context idea immediately legible to stakeholders.

Success criteria for the POC

People should feel that the board is central, the planner is helpful but bounded, and the same project context is visible to both humans and AI.